

Cours 28 — IA : les k plus proches voisins

Semaine 28 ■ Manuel Hachette ch. 13 (§ 13.1, 13.2) ■ 2×1 h

Objectifs

- Situer l'apprentissage automatique dans l'intelligence artificielle
- D  rouler et programmer l'algorithme des k plus proches voisins (KNN)
- Discuter le choix de k et les limites de la m  thode

1. Apprendre des exemples

Jusqu'ici, nos programmes appliquent des **r  gles**   crites par nous. L'**apprentissage automatique** (*machine learning*) inverse la d  marche : on fournit des **exemples d  j     tiquet  s**, et le programme en d  duit comment classer les cas nouveaux — reconnaître un chiffre manuscrit, filtrer un spam, recommander un film.

Vocabulaire sur un exemple : on veut reconnaître l'esp  ce d'un poisson    partir de sa longueur et de sa masse. Les donn  es d'**entra  nement** : une table (semaine 18!) d'individus d  crits par des **attributs** num  riques (longueur, masse) et une **  tiquette** (l'esp  ce, la **classe**).

2. L'algorithme des k plus proches voisins

Id  e : *dis-moi qui sont tes voisins, je te dirai qui tu es*. Pour classer un nouvel individu :

1. calculer sa **distance**    chaque exemple d'entra  nement — la distance euclidienne du TP 8 : $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$;
2. retenir les **k plus proches** (trier par distance — semaine 20 : **sorted** et sa cl  !) ;
3. la classe **majoritaire** parmi eux l'emporte (compter : le motif occurrences, semaine 10).

```
def knn(table, individu, k):
    """Renvoie la classe majoritaire des k plus proches
    voisins de individu dans table.

    Pr  conditions : k >= 1, k <= len(table), k impair conseill  .
    """
    voisins = sorted(table, key=lambda e: distance(e, individu))
    votes = {}
    for e in voisins[:k]:
        c = e["classe"]
        if c in votes:
            votes[c] = votes[c] + 1
        else:
            votes[c] = 1
    return classe_majoritaire(votes)
```

(**lambda e** : ... : une fonction cl   anonyme,   crite sur place — on peut aussi d  finir une fonction nomm  e comme en semaine 20.)

3. Choisir k

- **k trop petit** (k = 1) : hypersensible aux exemples aberrants — un intrus mal   tiquet   contamine son voisinage.

- **k trop grand** : on vote avec des individus lointains ; à la limite ($k = \text{tout le monde}$), on répond toujours la classe la plus nombreuse.
- **k impair** évite les égalités entre deux classes.

Comment choisir en pratique ? **Tester** : garder une partie des données étiquetées de côté (jeu de **test**), classer ces individus « comme s'ils étaient nouveaux », mesurer le taux de réussite pour plusieurs k — le jeu de tests de la semaine 13, version données.

4. Limites et précautions

- **Échelles** : si la masse va de 0 à 5000 g et la longueur de 0 à 50 cm, la masse écrase tout dans la distance — **normaliser** d'abord (TP 9 : **normalise!**).
- **Coût** : classer un individu = distance à *tous* les exemples ($O(n)$, puis tri...) — lent sur de grosses bases.
- **Biais** : l'algorithme reproduit les données ; des exemples biaisés donnent des réponses biaisées. Une IA n'est jamais meilleure que ses données.

À retenir

- Apprentissage automatique : des exemples étiquetés $\rightarrow$ classer du nouveau.
- KNN : distances, k plus proches, vote majoritaire ; k impair, ni trop petit ni trop grand.
- Évaluer sur un jeu de test réservé ; normaliser les attributs d'échelles différentes.
- Les biais des données deviennent les biais du classifieur.